

Transformer Architecture & Attention Mechanisms

A Deep Technical Reference for Large Language Models

Version 3.1 | Research Division | NeuralSystems Lab | 2024

Abstract. This document provides a comprehensive technical reference for the Transformer architecture, covering self-attention mechanics, positional encodings, feed-forward sublayers, training dynamics, scaling laws, architectural variants (encoder-only, decoder-only, encoder-decoder), and advanced topics such as Rotary Position Embeddings (RoPE), Grouped-Query Attention (GQA), mixture-of-experts (MoE) routing, and quantization strategies. Each section includes formulas, design rationale, benchmark comparisons, and known failure modes. This reference is intended for practitioners building or evaluating large language model systems.

Table of Contents

1.	Foundational Architecture Overview	2
2.	Tokenization & Embedding Layer	2
3.	Self-Attention: Scaled Dot-Product	3
4.	Multi-Head Attention (MHA)	4
5.	Positional Encodings: Sinusoidal vs. RoPE vs. ALiBi	5
6.	Feed-Forward Sublayer & Activation Functions	6
7.	Layer Normalization Variants	7
8.	Encoder-Only, Decoder-Only, and Encoder-Decoder Models	7
9.	Grouped-Query Attention (GQA) & Multi-Query Attention (MQA)	8
10.	Mixture of Experts (MoE)	9
11.	Training Dynamics & Scaling Laws	10
12.	Inference Optimizations: KV Cache & Quantization	11
13.	Benchmark Comparison Table	12
14.	Failure Modes & Known Limitations	13

1. Foundational Architecture Overview

The Transformer, introduced by Vaswani et al. (2017) in *Attention Is All You Need*, eliminated recurrence entirely in favor of attention mechanisms. A standard Transformer block consists of two sublayers: (a) a Multi-Head Self-Attention (MHSA) sublayer and (b) a position-wise Feed-Forward Network (FFN). Each sublayer is wrapped with a residual connection and a normalization step.

The original paper used Post-LayerNorm (Post-LN), where normalization is applied after the residual addition. Modern LLMs predominantly use Pre-LayerNorm (Pre-LN) because it produces more stable gradients at large scale, at the cost of slightly lower peak performance in small-scale settings.

Core equation per Transformer block (Pre-LN):

```
h' = x + MHSA(LayerNorm(x))
output = h' + FFN(LayerNorm(h'))
```

Component	Role	Typical Hyperparameters
Token Embedding	Map token IDs to dense vectors	vocab_size=32k-128k, d_model=512-8192
Positional Encoding	Inject position info	max_seq_len, RoPE base (default 10000)
Self-Attention	Model token relationships	n_heads=8-128, d_head=64-128
FFN	Non-linear transformation	expansion=4x or 8x d_model, SwiGLU/GELU
Layer Norm	Stabilize training	eps=1e-5 or 1e-6, RMSNorm preferred
Output Head	Project to vocabulary logits	weight-tied with embedding (often)

Table 1: Transformer component summary with typical hyperparameters.

2. Tokenization & Embedding Layer

Modern LLMs use subword tokenization algorithms — primarily Byte-Pair Encoding (BPE) or SentencePiece Unigram — to balance vocabulary coverage and sequence length. The tokenizer converts raw text into a sequence of integer token IDs, each mapped to a learned embedding vector of dimension d_{model} .

BPE Algorithm (Simplified):

1. Start with character-level vocabulary
2. Count all adjacent symbol pairs in training corpus
3. Merge the most frequent pair into a single token
4. Repeat for N merge operations ($N = \text{vocab_size} - \text{base_chars}$)
5. The final vocabulary is used at inference time via a greedy tokenization pass

The embedding matrix E has shape $[\text{vocab_size}, d_{model}]$. At inference, it acts as a lookup table. During training the embeddings are updated via backpropagation. Weight tying — sharing the embedding matrix with the final output projection — reduces parameter count significantly and often improves perplexity. GPT-2, LLaMA-1, and Mistral all use weight tying.

Note: tiktoken (used by GPT-4) uses BPE with byte-level fallback, ensuring that any Unicode string is tokenizable without an <UNK> token. LLaMA-3 expanded its vocabulary from 32,000 (LLaMA-2) to 128,256 tokens to better

cover multilingual and code content.

3. Self-Attention: Scaled Dot-Product

Scaled dot-product attention computes, for every query vector, a weighted sum of value vectors, where weights reflect query-key similarity. Given input matrix X of shape $[\text{seq_len}, d_{\text{model}}]$, we project to Q, K, V matrices:

$$Q = X W_Q, K = X W_K, V = X W_V \quad \text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V$$

The scaling factor $1/\sqrt{d_k}$ (where d_k is the key dimension, typically 64 or 128) prevents the dot products from growing too large in magnitude, which would push the softmax into regions of near-zero gradient. Without scaling, training with large d_k becomes unstable.

Causal (Decoder) Masking:

In autoregressive decoders, a lower-triangular mask M is applied before the softmax to prevent each position from attending to future tokens: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}} + M\right) V$ where $M[i, j] = 0$ if $j \leq i$, else $-\infty$

The computational cost of full self-attention is $O(n^2 * d)$ in time and $O(n^2)$ in memory for a sequence of length n . This quadratic scaling is the primary bottleneck for very long contexts and has motivated numerous approximations: sparse attention, linear attention, sliding window attention (Longformer), and state-space models (Mamba).

Variant	Time Complexity	Memory	Supports Causal?	Notes
Full Attention	$O(n^2 d)$	$O(n^2)$	Yes	Baseline; exact
Sparse (fixed)	$O(n \sqrt{n} \cdot d)$	$O(n \sqrt{n})$	Yes	BigBird, Longformer
Linear Attention	$O(n d^2)$	$O(n d)$	Limited	Approx.; quality loss
Flash Attention	$O(n^2 d)$	$O(n)$	Yes	IO-aware; same math
Sliding Window	$O(n \cdot w \cdot d)$	$O(n \cdot w)$	Yes	w =window; Mistral
State Space (Mamba)	$O(n \cdot d)$	$O(d)$	Yes	Recurrent; diff. paradigm

Table 2: Attention variants — complexity and characteristics.

4. Multi-Head Attention (MHA)

Rather than applying a single attention function, MHA runs h independent attention heads in parallel, each projecting into a lower-dimensional subspace of size $d_{\text{head}} = d_{\text{model}} / h$. The outputs are concatenated and projected back:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O \quad \text{where } \text{head}_i = \text{Attention}(Q W_{Q_i}, K W_{K_i}, V W_{V_i})$$

$W_{Q_i}, W_{K_i}, W_{V_i} \in \mathbb{R}^{(d_{\text{model}} \times d_{\text{head}})} \quad W_O \in \mathbb{R}^{(h \cdot d_{\text{head}} \times d_{\text{model}})}$

Different heads learn to attend to different types of relationships: syntactic dependencies, coreference, positional proximity, semantic similarity, etc. Probing studies show that heads in lower layers tend to capture positional patterns while upper-layer heads capture more semantic structure.

Head Ablation Findings (from literature):

- Removing up to 20% of heads in BERT causes < 1% drop in downstream task accuracy.
- A small number of 'induction heads' (typically in layers 1–4) are critical for in-context learning.
- Attention sink phenomenon: the first token (often BOS or padding) accumulates a disproportionate share of attention, acting as a 'global sink'. StreamingLLM exploits this.
- In GPT-style decoders, certain heads specialize in copying recent tokens — these are essential for few-shot prompting.

5. Positional Encodings: Sinusoidal vs. RoPE vs. ALiBi

Attention is permutation-invariant by default. Positional encodings inject order information so the model can distinguish 'the cat sat on the mat' from 'the mat sat on the cat'. Three major families exist:

5.1 Sinusoidal (Absolute)

```
The original Transformer adds fixed sinusoidal signals to embeddings before the first
layer: PE(pos, 2i) = sin(pos / 10000^(2i/d_model)) PE(pos, 2i+1) = cos(pos /
10000^(2i/d_model))
```

Advantages: no learned parameters, generalizes trivially to any length. Disadvantages: position information is mixed into the residual stream from the start and degrades for sequences beyond the training length.

5.2 Rotary Position Embedding (RoPE)

```
RoPE (Su et al., 2021), used in LLaMA, Mistral, Falcon, and most modern LLMs, encodes
position by rotating query and key vectors before the dot product. For a 2D case:
R(theta) = [[cos(theta), -sin(theta)], [sin(theta), cos(theta)]] The dot product of a
rotated Q at position m and rotated K at position n depends only on the relative offset
(m - n), giving the model a natural relative-position signal without explicit relative
attention.
```

RoPE's base frequency (default 10,000 for LLaMA-1/2) can be increased to extend context. LLaMA-3 uses a base of 500,000 and YaRN scaling to support 128K context. Code Llama uses base 1,000,000.

5.3 ALiBi (Attention with Linear Biases)

```
ALiBi adds a static linear penalty to attention logits proportional to distance,
requiring no positional embedding at all: Score(i, j) = Q_i · K_j / sqrt(d_k) - m * |i -
j| where m is a per-head scalar slope (set geometrically across heads).
```

ALiBi extrapolates well beyond training length and is used in MPT and BLOOM. However, it tends to underperform RoPE on tasks requiring long-range semantic retrieval because the linear penalty biases strongly toward local context.

Property	Sinusoidal	RoPE	ALiBi
Learned params	No	No	No
Relative position	Implicit	Yes (exact)	Yes (linear)
Length generalization	Poor	Medium (with scaling)	Good

Integration point	Input layer	Per-layer QK	Per-layer logits
Used in	Original Transformer	LLaMA, Mistral, Falcon	MPT, BLOOM
Extrapolation strategy	N/A	YaRN, LongRoPE	None needed

Table 3: Positional encoding family comparison.

6. Feed-Forward Sublayer & Activation Functions

Each Transformer layer includes a two-layer MLP applied position-wise. The intermediate dimension is typically $4 \times d_{\text{model}}$ (e.g., 4096 for $d_{\text{model}}=1024$):

$$\text{FFN}(x) = \text{activation}(x W_1 + b_1) W_2 + b_2$$

Standard: W_1 in $\mathbb{R}^{(d_{\text{model}} \times d_{\text{ff}})}$, W_2 in $\mathbb{R}^{(d_{\text{ff}} \times d_{\text{model}})}$

Gated Linear Units (SwiGLU):

LLaMA and most modern LLMs replace the standard FFN with SwiGLU, which uses three weight matrices and a multiplicative gate: $\text{SwiGLU}(x) = (x W_{\text{gate}} * \text{SiLU}(x W_1)) W_2$
 $\text{SiLU}(x) = x * \text{sigmoid}(x)$ [also called Swish]
 To keep total parameter count constant, d_{ff} is typically reduced to $\frac{8}{3} * d_{\text{model}}$ (instead of $4x$) when using SwiGLU.

SwiGLU outperforms ReLU and GELU on language modeling benchmarks by 0.5–1.5 perplexity points across model sizes. The smooth, non-monotonic nature of Swish allows gradients to flow more uniformly through the network during training.

Activation	Formula	Used In	Ppl Advantage vs ReLU
ReLU	$\max(0, x)$	GPT-2, early BERT	baseline
GELU	$x * \Phi(x)$	GPT-3, BERT-large	+0.3-0.5
SiLU/Swish	$x * \text{sigmoid}(x)$	Various	+0.4-0.7
SwiGLU	$\text{Swish}(xW) * (xV)$	LLaMA 1/2/3, Mistral, Gemma	+0.5-1.5
GeGLU	$\text{GELU}(xW) * (xV)$	PaLM, T5-XXL variants	+0.4-1.2

Table 4: Activation function comparison for FFN sublayers.

7. Layer Normalization Variants

Layer normalization (LayerNorm) normalizes activations across the feature dimension. Two dominant variants are used in LLMs:

$$\text{LayerNorm: } y = \gamma * (x - \text{mean}(x)) / \sqrt{\text{var}(x) + \epsilon} + \beta \quad \text{RMSNorm: } y = \gamma * x / \sqrt{\text{mean}(x^2) + \epsilon}$$

RMSNorm (Root Mean Square LayerNorm) removes the mean-centering step, reducing computation by ~15% while achieving comparable or superior training stability. It is used in LLaMA, Mistral, Falcon, and Gemma. The γ (gain) parameter is learned; RMSNorm omits the β (bias) parameter entirely.

8. Encoder-Only, Decoder-Only, and Encoder-Decoder Models

Architecture	Attention Type	Primary Use Cases	Examples
Encoder-only	Bidirectional (full)	Classification, NER, embeddings, semantic search	BERT, RoBERTa, DeBERTa, ModernBERT
Decoder-only	Causal (masked)	Text generation, coding, reasoning, chat	GPT-4, LLaMA 3, Mistral, Gemma, Claude
Encoder-Decoder	Bidirectional + Cross-Attention	Translation, summarization, QA with long context	TextBART, mT5, Flan-T5

Table 5: Transformer architecture families.

The decoder-only paradigm has become dominant for large-scale general-purpose models. A key reason is the simpler pre-training objective (next-token prediction, or causal language modeling) which scales effectively and yields emergent few-shot capabilities. Encoder-only models still lead on embedding and discriminative tasks due to bidirectional context access. Encoder-decoder models remain competitive for structured generation tasks.

9. Grouped-Query Attention (GQA) & Multi-Query Attention (MQA)

Standard MHA uses h separate K, V projection matrices (one per head). Multi-Query Attention (MQA, Shazeer 2019) uses a single shared K and V across all heads. Grouped-Query Attention (GQA, Ainslie et al. 2023) is a middle ground: G groups share K and V matrices, where $1 \leq G \leq h$.

MHA: h query heads, h key heads, h value heads $\rightarrow h$ KV pairs cached MQA: h query heads, 1 key head, 1 value head $\rightarrow 1$ KV pair cached GQA: h query heads, G key heads, G value heads $\rightarrow G$ KV pairs cached Example: LLaMA-3-70B uses $h=64$ query heads, $G=8$ key/value heads
The KV cache at inference stores K and V tensors for all past tokens. Its size is: KV cache bytes = $2 * n_layers * n_kv_heads * d_head * seq_len * bytes_per_param$ For LLaMA-3-70B (FP16) at 128K context: $2 * 80 * 8 * 128 * 131072 * 2 = \sim 43$ GB vs. MHA equivalent: $2 * 80 * 64 * 128 * 131072 * 2 = \sim 344$ GB

GQA reduces KV cache size by a factor of h/G with minimal quality degradation. Models using GQA: LLaMA-2-70B ($G=8$), LLaMA-3 all sizes, Mistral-7B ($G=8$), Gemma, Falcon-180B. MQA is used in PaLM and Gemini.

10. Mixture of Experts (MoE)

Mixture-of-Experts models replace the dense FFN in some or all layers with a set of E expert FFNs, only K of which are activated for each token ($K \ll E$, typically $K=1$ or $K=2$). A learned router (a small linear layer + softmax) assigns tokens to experts:

Router output: $g = \text{softmax}(x W_{\text{router}})$ [shape: E] Top- K selection: select indices $i_1..i_K = \text{top}_k(g)$ MoE output: $\sum_{j=1}^K g[i_j] * \text{FFN}_{i_j}(x)$

Sparse MoE dramatically increases parameter count without proportionally increasing compute: Mixtral 8x7B has ~ 47 B total parameters but activates only ~ 13 B per forward pass. GPT-4 is widely believed to use a MoE architecture (though unconfirmed).

MoE Challenges:

- Load imbalance: without auxiliary losses, the router collapses to always selecting the same few experts (expert collapse). Switch Transformer uses an auxiliary load-balancing loss.

- Communication overhead: in distributed training, MoE requires all-to-all communication across devices when experts are sharded, adding significant latency.
- Expert routing granularity: token-level routing (most common) can cause the same concept to be split across experts mid-sentence. Some proposals use sentence-level or segment-level routing.
- Fine-tuning instability: MoE models can be harder to fine-tune than dense models because gradient updates are sparser and expert specialization can break.

Model	Experts (E)	Active/token (K)	Total Params	Active Params
Switch-Base	128	1	7.4B	~0.2B
Mixtral 8x7B	8	2	46.7B	12.9B
Mixtral 8x22B	8	2	~141B	~39B
Grok-1 (est.)	8	2	314B	~86B
DeepSeek-V2	160	6	236B	21B
DeepSeek-V3	256	8	671B	37B

Table 6: Mixture-of-Experts model specifications.

11. Training Dynamics & Scaling Laws

Kaplan et al. (2020) established that LLM loss follows power-law scaling with model size (N), dataset size (D), and compute budget (C):

```
L(N) ~ (N_c / N)^(alpha_N), alpha_N ~ 0.076 L(D) ~ (D_c / D)^(alpha_D), alpha_D ~ 0.095
L(C) ~ (C_c / C)^(alpha_C), alpha_C ~ 0.050 Optimal allocation: N ~ C^0.73, D ~ C^0.27
(Kaplan) Chinchilla (Hoffmann 2022): D ~ 20 * N (more data-efficient)
```

The Chinchilla scaling laws revised the Kaplan prescription significantly: GPT-3 (175B params, ~300B tokens) was undertrained relative to compute-optimal; the same compute budget should train a ~70B model on ~1.4T tokens. LLaMA-1 was deliberately trained beyond compute-optimal to produce a smaller, inference-efficient model. LLaMA-3 used 15T+ tokens for its 8B model.

Training Stability Techniques:

- Gradient clipping: clip global gradient norm to 1.0 to prevent exploding gradients, especially early in training.
- Learning rate schedule: linear warmup for 1-2% of total steps, followed by cosine decay to 10% of peak LR. Peak LR ~ 3e-4 for small models, 1e-4 for large.
- Weight decay: AdamW with weight decay 0.1 applied to all non-embedding, non-normalization parameters.
- Mixed precision: BF16 (not FP16) is preferred for large models because BF16 has the same exponent range as FP32, preventing overflow without loss scaling.
- Z-loss regularization (PaLM): adds a small penalty on the log-partition function of the router softmax to prevent logit divergence.

12. Inference Optimizations: KV Cache & Quantization

KV Cache:

During autoregressive generation, the K and V tensors for all past tokens are cached and reused. Only the new token's Q, K, V are computed at each step, reducing computation from $O(n^2)$ to $O(n)$ per step (at the cost of $O(n)$ memory per layer). Paged KV cache (vLLM's PagedAttention) manages cache memory in fixed-size blocks, enabling high GPU utilization for variable-length batches.

Quantization Strategies:

Quantization reduces the bit-width of weights and/or activations to save memory and increase throughput at the cost of some accuracy.

Method	Weights	Activations	Speedup	Quality Loss	Notes
FP16/BF16	16-bit	16-bit	1x	None (baseline)	Default training dtype
INT8 (LLM.int8)	8-bit	16-bit	~1.5x	< 0.1%	Mixed decomposition
GPTQ	4-bit	16-bit	~2.5x	~0.5-1%	Post-training, per-layer
AWQ	4-bit	16-bit	~2.5x	~0.3-0.8%	Activation-aware
GGUF Q4_K_M	4-bit mixed	32-bit	~3x	~0.7%	llama.cpp default
FP8 (H100)	8-bit float	8-bit float	~2x	< 0.2%	Hardware native

INT4 + INT8 act	4-bit	8-bit	~3.5x	~1-2%	SmoothQuant variant
-----------------	-------	-------	-------	-------	---------------------

Table 7: Quantization methods for LLM inference.

13. Benchmark Comparison Table

The following table summarizes architectural choices and reported benchmark scores for major open and proprietary LLMs. Scores are from official reports or the Open LLM Leaderboard (as of early 2024). MMLU = Massive Multitask Language Understanding (5-shot); HumanEval = code generation (pass@1); GSM8K = grade-school math (8-shot CoT).

Model	Params	Attn	PosEnc	Norm	Act	Context	MMLU	HumanEval	GSM8K
GPT-3	175B	MHA	Learned	Post-LN	GELU	2K	43.9	—	—
PaLM	540B	MQA	RoPE	Pre-LN	SwiGLU	2K	69.3	26.2	56.5
LLaMA-1 65B	65B	MHA	RoPE	RMSNorm	SwiGLU	2K	63.4	23.7	50.9
LLaMA-2 70B	70B	GQA(8)	RoPE	RMSNorm	SwiGLU	4K	68.9	29.9	57.5
Mistral-7B	7.3B	GQA(8)	RoPE	RMSNorm	SwiGLU	32K	60.1	30.5	52.1
Mixtral 8x7B	46.7B	GQA(8)	RoPE	RMSNorm	SwiGLU	32K	70.6	40.2	74.4
LLaMA-3 70B	70B	GQA(8)	RoPE	RMSNorm	SwiGLU	128K	82.0	72.6	93.0
Gemma-2 27B	27B	GQA(2)	RoPE	RMSNorm	GeGLU	8K	75.2	51.8	74.0
DeepSeek-V3	671B MoE	MLA	RoPE	RMSNorm	SwiGLU	128K	88.5	65.2	89.3

Table 8: Architectural choices and benchmark scores for key LLMs. MLA = Multi-head Latent Attention (DeepSeek’s low-rank KV compression variant).

14. Failure Modes & Known Limitations

Despite impressive capabilities, Transformer LLMs exhibit systematic failure modes that are important to understand when building applications:

- Hallucination:** Models generate fluent but factually incorrect text. Root causes include spurious memorization of co-occurring tokens and inability to distinguish 'what I have seen' from 'what is true'. Retrieval-Augmented Generation (RAG) partially mitigates this.
- Lost-in-the-Middle:** Liu et al. (2023) showed that LLMs perform best when relevant information is at the beginning or end of a long context, and significantly worse when it is in the middle — even within the nominal context window.
- Attention Sink & Long-Context Degradation:** Beyond fine-tuned context lengths, RoPE-based models assign high attention scores to position 0 (sink token) and fail to meaningfully attend to distant tokens. StreamingLLM and positional interpolation techniques address this.
- Reversal Curse:** Berglund et al. (2023) showed that a model trained on 'A is B' fails to infer 'B is A', indicating that causal LMs learn directional associations rather than symmetric factual relationships.
- Repetition Collapse:** Without repetition penalties, greedy and beam-search decoding can collapse into degenerate loops. Temperature sampling, top-p, and frequency penalties are standard mitigations. The

nucleus (top-p) technique discards low-probability tokens to maintain diversity without sacrificing coherence.

Sensitivity to Prompt Format: Minor reformulations of the same question can lead to vastly different answers — a form of brittleness not seen in human reasoning. Chain-of-Thought prompting significantly improves reasoning robustness.

Tokenization Artifacts: Arithmetic on numbers is unreliable because digits are tokenized inconsistently (e.g., '1234' may be one token while '1235' is two). This creates systematic biases in numeric tasks.

Document end. This reference covers foundational and advanced Transformer topics as of 2024. Topics not covered here include RLHF/RLAIF alignment training, speculative decoding, model merging techniques, continuous batching, and multi-modal extensions.